DZIENNIK UWB · MINI-PROJEKT M1

Rekurencja krok po kroku

Interaktywne laboratorium wywołań, przypadków bazowych i powrotów

Kod tematu	**ASD1-M1-14**
Autor / student	**Edvin Zenevič**
Numer indeksu	**93783**
Przedmiot	**Algorytmy i struktury danych I**
Rok akademicki	**2026/2027 · semestr I**
Data opracowania	**10 października 2026**

Dokumentacja techniczna i dydaktyczna · 10 stron

Aplikacja: index.html · Slajdy: prezentacja/prezentacja.html
Kolejność obrony: prezentacja → produkt → pytania

02 / Cel i zakres projektu

Problem dydaktyczny

Rekurencja bywa błędnie rozumiana jako „funkcja powtarzająca samą siebie”. Kluczowe jest to, że każde wywołanie ma własne argumenty i czeka na wynik wywołania potomnego. Laboratorium pozwala obserwować oba kierunki pracy: zejście do przypadku bazowego oraz powroty, podczas których powstaje wynik. Odbiorcą jest student poznający rekurencję; celem jest samodzielne przewidywanie następnego kroku, a nie tylko oglądanie animacji.

Zakres zgodny z kartą [1]

Moduł	Dane	Przypadki bazowe
Suma przez podział	0–16 liczb całkowitych −20…20	Pusty fragment → 0; jeden element → jego wartość
Potęgowanie zwykłe i szybkie	Podstawa −10…10; wykładnik 0…10	n = 0 → 1, także 0^0; szybki także n = 1 → a
Wieże Hanoi	0–8 krążków, słupki A, B, C; cel A → C	n = 0 → zero ruchów

Wykonane funkcje

Jedna aplikacja zawiera wszystkie moduły. Użytkownik podaje własne dane i wybiera trzy scenariusze w każdym module. Sterowanie obejmuje Krok, Start/Pauza, Reset i tempo 100–2000 ms. Zmiana danych unieważnia poprzedni przebieg. Widoczne są: problem, aktywne ramki, argumenty, zmienne lokalne, drzewo z wynikami, bieżący wiersz pseudokodu i uzasadnienie zdarzenia.

Tryb przewidywania zatrzymuje pokaz przed wejściem lub powrotem. Pytanie zależy od rzeczywistego stosu. Dodatkowe trzy pytania mają odpowiedzi i wyjaśnienia. Tabela porównuje wyłącznie dwa warianty potęgowania na identycznych danych. Testy można uruchomić z aplikacji lub z Node.js.

Warunki użycia i kryteria poprawności

Program działa po otwarciu index.html w lokalnej przeglądarce, bez kont i płatnych usług. Nie wysyła danych do sieci. Wynik sumy i potęgi musi być zgodny z obliczeniami kontrolnymi; Hanoi kończy na C bez naruszenia kolejności krążków. Po ostatnim powrocie stos jest pusty, lecz drzewo zachowuje zakończone wywołania.

> Nie utożsamiamy stosu aktywnych wywołań z całym drzewem. Pierwszy opisuje „co jeszcze pracuje”, drugi „co zostało wywołane”.

03 / Algorytmy i mechanizmy

Suma przez podział

Funkcja otrzymuje indeksy lo i hi opisujące półotwarty przedział [lo, hi). Dla lo = hi zwraca 0; dla hi − lo = 1 zwraca tablica[lo]. W pozostałych przypadkach mid = ⌊(lo + hi)/2⌋. Lewa i prawa połowa są obliczane kolejno, a dopiero po obu powrotach dodawane. Badany algorytm nie tworzy kopii połówek: kopia fragmentu występuje tylko w danych zdarzenia do wizualizacji.

Dla [2,4,1,3] liście zwracają 2, 4, 1 i 3. Połączenia dają 6 i 4, a korzeń 10. Poprawność wynika z rozłączności połówek i pokrycia całego przedziału. Każde wywołanie dla większego fragmentu zmniejsza jego długość.

Dwa warianty potęgowania

Wariant zwykły wykorzystuje $a^n = a \cdot a^{n-1}$. Jedyną bazą jest n = 0. Wariant szybki ma bazy n = 0 i n = 1, a dla większego n wywołuje funkcję raz dla ⌊n/2⌋ i zachowuje wynik half. Oblicza half · half; gdy n jest nieparzyste, dopisuje jeszcze czynnik a. Zwrócona wartość half jest użyta ponownie bez ponownego wywołania.

> Dla 2^5: szybka ścieżka n = 5 → 2 → 1; powroty: 2 → 4 → 32. Są 3 mnożenia i maksymalnie 3 ramki. Zwykła ścieżka schodzi do 0: 5 mnożeń, 6 ramek.

Zmniejszenie wykładnika gwarantuje zakończenie. Tożsamości $a^{2k} = (a^k)^2$ i $a^{2k+1} = (a^k)^2 \cdot a$ uzasadniają poprawność również dla ujemnej podstawy. $0^0 = 1$ jest konwencją zadania, nie deklaracją uniwersalnej definicji matematycznej.

Wieże Hanoi

Dla n > 0 funkcja przenosi n − 1 krążków z from na aux, następnie krążek n z from na to, a w końcu n − 1 z aux na to. Argumenty słupków zmieniają rolę w wywołaniach dzieci. Ruch ma identyfikator ramki, która go wykonała. Dla n = 0 emitowany jest przypadek bazowy i powrót z liczbą 0, ale nie ma ruchu.

Zakładając poprawne rozwiązanie dla n − 1, największy rozważany krążek jest odsłonięty, a cel legalny. Minimalna liczba ruchów spełnia M(n) = 2M(n − 1) + 1, M(0) = 0. Walidator sprawdza top źródła, top celu oraz cały stan po ruchu. Podejście odpowiada klasycznemu rozkładowi rekurencyjnemu [2], lecz kod i wizualizacja są realizacją projektu [3].

04 / Struktury danych i złożoność

Model zdarzenia i stan odtwarzania

Każde zdarzenie zawiera type, id, parentId, depth, name, args, line i reason. Zależnie od typu dochodzą locals, value lub move = {disk, from, to}. Id jest unikatowe w obrębie przebiegu; parentId wiąże wywołanie z rodzicem. depth korzenia wynosi 0. Liczba aktywnych ramek wynosi depth + 1 przy wejściu do danego wywołania.

Stos to tablica: push przy wejściu, pop przy powrocie. Drzewo to Map indeksowana id z parentId i statusem active/finished; zakończony węzeł nie znika. Słupki Hanoi są trzema tablicami od dołu do góry: koniec tablicy oznacza górny krążek. Zmienna cursor wskazuje następne nieodtworzone zdarzenie. Zapisujemy jeden bieżący stan, nie kolekcję klatek.

Metoda	Czas obliczeń	Pamięć badanego algorytmu
Suma, $n \geq 1$	$\Theta(n)$, $n - 1$ dodawań	Stos $\Theta(\log n)$; wejście $\Theta(n)$
Potęga zwykła	$\Theta(n + 1)$, n mnożeń	Stos $\Theta(n + 1)$
Potęga szybka, $n \geq 2$	$\Theta(\log n)$	Stos $\Theta(\log n)$
Hanoi	$\Theta(2^n)$, $2^n - 1$ ruchów	Stos $\Theta(n + 1)$ i słupki $\Theta(n)$

Tabela 1. Koszty bez instrumentacji; operacje na ograniczonych liczbach traktujemy jako O(1).

Definicje liczników

calls liczy każde wejście, również bazowe. maxDepth liczy największą liczbę aktywnych ramek, nie numer poziomu. multiplications liczy wyłącznie wykonane operacje * w potęgowaniu; additions — połączenia sum; moves — legalne przeniesienia Hanoi. W widoku pokazujemy liczniki odtworzonego prefiksu, w porównaniu — całego przebiegu. Dla pustych danych baza nadal wymaga jednej ramki.

Narzut wizualizacji liczony oddzielnie

Nie mierzymy czasu renderowania jako czasu algorytmu. Rejestr zdarzeń i węzły wymagają O(E) rekordów, gdzie E to liczba zdarzeń. Przy sumie zapis fragmentów daje łącznie O(n log n) skopiowanych liczb. Odtwarzanie przebudowuje do O(E) węzłów DOM na krok, więc cały pokaz może kosztować $O(E^2)$. Argumenty i nazwy w Hanoi mają stałą wielkość, ale walidacja pełnego układu ma O(n) na ruch i Set o O(n) pamięci.

Ograniczenia z karty utrzymują narzut na dydaktycznie użytecznym poziomie. To laboratorium śledzenia wykonania, nie biblioteka do obliczeń wysokowydajnych. Granice inputu są walidowane przed generowaniem śladu.

05 / Technologie i uruchomienie

Wybór technologii

HTML definiuje semantyczne sekcje, formularze i tabele. CSS odpowiada za układ responsywny, kontrast i legendę. JavaScript zawiera badane algorytmy, odtwarzacz i obsługę interfejsu. Aplikacja nie używa bibliotek interfejsu, bundlera, CDN ani API. Dzięki klasycznym skryptom zamiast importów modułów można otworzyć index.html z file:// bez serwera.

Algorytmy oraz testy udostępniają te same funkcje przeglądarce i Node.js. Ograniczenie $|a| \leq 10$, $n \leq 10$ daje najwyżej 10^{10}, czyli wartość poniżej Number.MAX_SAFE_INTEGER. Nie potrzebujemy BigInt. Dla większych zakresów należałoby ponownie sprawdzić precyzję i model kosztu arytmetyki.

Uruchomienie aplikacji i testów

1. Rozpakuj archiwum, zachowując strukturę folderów.
2. Otwórz index.html w aktualnej przeglądarce, np. Firefox, Chrome lub Edge.
3. Wybierz moduł, dane i przycisk Krok albo Start.
4. Przycisk „Uruchom testy lokalnie” uruchamia tę samą podstawową serię 44 testów bez instalacji.
5. W terminalu, mając Node.js, wykonaj `node tests/run.cjs`. Opcja `--save` odświeża pliki wyniki/testy.json, testy.txt (po przekierowaniu wyjścia) i porownanie.json.

Narzędzia weryfikacji i dokumentacji

Node.js uruchamia testy. Playwright steruje Chromium w dodatkowej serii testów interfejsu: kliknięcia, zmiana pól, pytania, tryb bez sieci, ekran mobilny. Ten zestaw wymaga pomocniczej instalacji Playwright; nie jest wymagany dla odbiorcy aplikacji. Skrypt tests/browser.cjs odtwarza weryfikację i zapisuje zrzuty z rzeczywistego widoku.

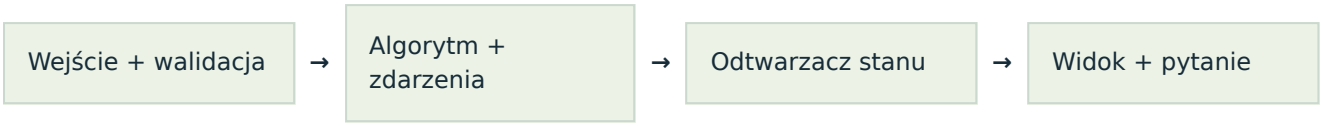
Python 3 przygotowuje źródła dokumentów HTML. Chromium eksportuje dokumentację i slajdy do PDF. pdf-lib sprawdza liczbę stron. To wyłącznie narzędzia wytwarzania, nie zależności produktu. Sposób odtworzenia podaje README.

Pliki do obrony

prezentacja/prezentacja.html i **prezentacja/prezentacja.pdf** zawierają dokładnie 10 slajdów. **dokumentacja/dokumentacja.pdf** zawiera 10 stron. HTML raportu jest źródłem edytowalnym, nie zamiennikiem wymaganego PDF. Do demonstracji służy index.html; wyniki testów dostępne są w aplikacji i folderze wyniki.

> Brak internetu nie blokuje aplikacji, testów w przeglądarce ani slajdów. Link do materiału Princeton jest źródłem do nauki, nie zasobem ładowanym przez produkt.

06 / Architektura i realizacja



Rysunek 1. Przepływ danych: obliczenia są oddzielone od odtwarzania i DOM.

Plik	Odpowiedzialność
app/algorithms.js	Walidacja, rekurencja, zdarzenia, metryki, legalność Hanoi, porównanie
app/playback.js	Stan, stos, drzewo, odtwarzanie jednego zdarzenia, pytanie zależne od stanu
app/content.js	Polskie opisy, zastosowania, ograniczenia i pseudokod
app/ui.js + app/styles.css	Formularze, scenariusze, zegar, renderowanie, odpowiedzi i testy
tests/suite.js + tests/run.cjs	Wspólne testy domenowe oraz zapis wyników w Node.js
tests/browser.cjs	Weryfikacja prawdziwego interfejsu i zrzuty
narzedzia/	Generacja źródeł i eksport dokumentów

Rzeczywisty ślad zamiast ręcznej animacji

createTrace wykonuje funkcje rekurencyjne synchronicznie i zapisuje zdarzenia w miejscu faktycznych operacji. Później advance odtwarza pojedynczy prefiks śladu. Animacja nie wykonuje ponownie rekurencji ani nie korzysta z nagrania. Odtwarzacz kontroluje, czy zdarzenie dotyczy szczytu stosu i czy rodzic nowej ramki jest aktualnym wywołaniem. Takie rozdzielenie umożliwia niezależny test obliczeń oraz stanu po dowolnym zdarzeniu.

Etapy wykonania

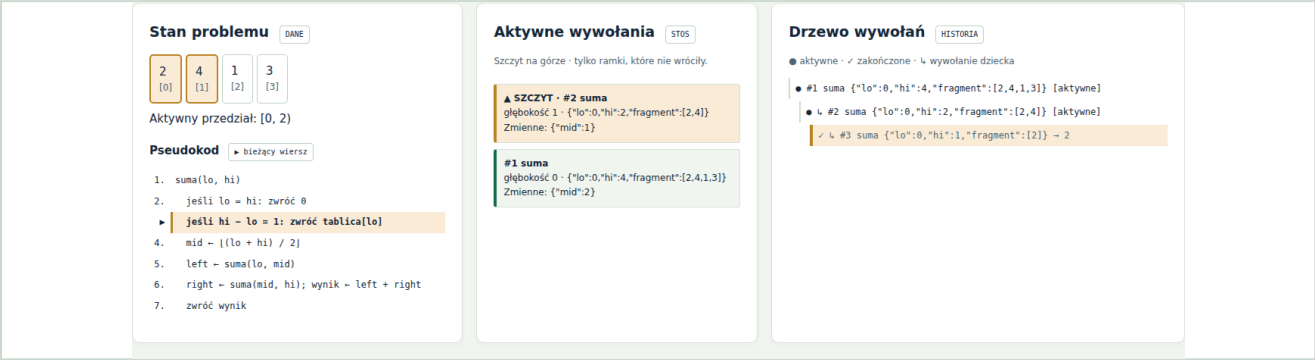
Najpierw ustalono zakres i model zdarzenia. Potem zaimplementowano wszystkie metody i walidator Hanoi. Następnie powstał odtwarzacz, polskie opisy i interfejs. Testy bazowe, końcowe i pośrednie uruchomiono przed stworzeniem dokumentów. Kolejny etap obejmował automatyczne testy przeglądarki oraz zrzuty. Na końcu opracowano porównanie, slajdy i raport na podstawie rzeczywistych danych.

Obsługa nowego przebiegu i błędów

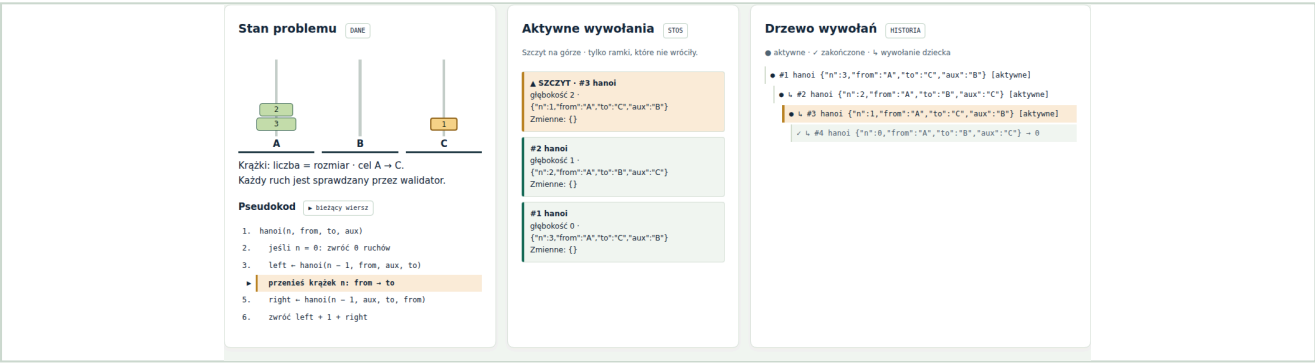
Zmiana wejścia zatrzymuje timer i czyści pytanie. Poprawne dane tworzą nowy ślad i pusty stan. Niepoprawne dane unieważniają stary ślad, blokują Krok/Start i wyświetlają przyczynę; nie pokazujemy starego wyniku jako aktualnego. Reset zachowuje wejście, lecz czyści odtworzenie. Tempo zmienia tylko harmonogram odtwarzania, nie wynik czy liczniki algorytmu.

Interfejs nie zna reguł rekurencji: pobiera stan i treść z osobnych modułów. Dzięki temu poprawność nie zależy od szybkości animacji ani rozmiaru ekranu.

07 / Interfejs i obsługa



Rysunek 2. Rzeczywisty widok po powrocie #3 dla [2,4,1,3]. Ramka #3 zwróciła 2 i znikła ze stosu; pozostały #2 oraz #1. W drzewie wynik #3 jest zachowany.



Rysunek 3. Pierwszy ruch Hanoi dla n = 3: krążek 1 trafia z A na C. Węzeł #3 wiąże ruch z odpowiednim wywołaniem; aktywny stos ma trzy ramki.

Przebieg demonstracji

Przyciski modułów przełączają problem. W scenariuszu kontrolnym Krok pokazuje jedno zdarzenie, a Start kontynuuje według wybranego tempa. Pauza przerywa zegar bez zmiany stanu. Reset wraca przed pierwsze wejście. Gdy pokaz jest zakończony, Krok i Start są nieaktywne; można zresetować albo zmienić dane.

Legenda i dostępność

▲ oznacza szczyt stosu, ● aktywne wywołanie, ✓ zakończone, ↳ dziecko, ▶ aktualny wiersz pseudokodu. Zakończenie, ruch i wynik mają opis tekstowy. Różnice nie są kodowane wyłącznie kolorem. Formularze mają etykiety, przyciski są dostępne z klawiatury, a błędy wejścia są ogłaszane przez role=alert. Na wąskim ekranie kolumny przechodzą do układu pionowego, a duże tabele mają własne przewijanie.

Pytanie o aktualny stan

Włączenie „Zatrzymuj przed wejściem / powrotem” powoduje pytanie przed wykonaniem zdarzenia. Dla wejścia trzeba przewidzieć liczbę ramek; dla powrotu — przyszły szczyt stosu. Błędna odpowiedź nie odblokowuje kroku. Poprawna ujawnia wyjaśnienie i pozwala wykonać dokładnie przewidziane zdarzenie. Nie przeskakujemy automatycznie przez pytanie.

08 / Porównanie potęgowania

Metoda pomiaru

Porównanie wykonano dla podstawy 2 i wykładników 1, 4, 8, 10. Każda komórka pochodzi z pełnego wykonania createTrace, a nie z pomiaru czasu animacji. Rozmiar wejścia oznacza wartość wykładnika n. Przy nieograniczonych liczbach należałoby odróżnić tę miarę od długości bitowej oraz uwzględnić koszt mnożenia wielkich liczb.

n	2^n	Mnożenia zwykłe	Mnożenia szybkie	Stos zwykły	Stos szybki
1	2	1	0	2	1
4	16	4	2	5	3
8	256	8	3	9	4
10	1024	10	4	11	4

Tabela 2. Rzeczywiste wyniki zapisane w wyniki/porownanie.json [3]. Stos oznacza liczbę ramek, korzeń = 1.

Interpretacja

Oba warianty dają identyczny wynik. Zwykły wykonuje dokładnie n mnożeń i przechowuje n + 1 ramek, gdyż schodzi do wykładnika 0. Szybki wykorzystuje dodatkową bazę n = 1. Dla potęg dwójki (4 i 8) każdy poziom nad bazą potrzebuje tylko kwadratu. Dla n = 10 ścieżka wykładników to 10 → 5 → 2 → 1: występują trzy kwadraty oraz jedno dodatkowe mnożenie dla wykładnika 5.

Nieparzyste wykładniki wpływają na liczbę mnożeń, ale nie wydłużają ścieżki względem połowienia. Ogólnie dla n ≥ 1 liczba mnożeń szybkiego wariantu wynosi $\lfloor \log_2 n \rfloor + \text{popcount}(n) - 1$. popcount to liczba jedynek w zapisie binarnym. Dla n = 10 (1010_2) mamy 3 + 2 − 1 = 4.

> Dla 2^1 wynik obu metod to 2. Zwykła mnoży 2 · 1, szybka od razu zwraca 2. Dlatego w pierwszym wierszu występuje 1 wobec 0, a nie 1 wobec 2.

Czego nie wynika z tabeli

Nie jest to benchmark czasu ściennego. W małym zakresie narzut zapisu zdarzeń i renderowania może dominować, a różnica kilku mnożeń nie musi być widoczna w czasie zegarowym. Liczniki pokazują pracę algorytmiczną. Nie sumujemy mnożeń, dodawań i ruchów w jeden ranking: odnoszą się do różnych problemów i innych jednostek pracy.

Porównanie z przykładem kontrolnym

Dla 2^5, użytego w aplikacji, wyniki wynoszą 32. Zwykły wariant ma 5 mnożeń i 6 ramek, szybki 3 mnożenia i 3 ramki. Dla sumy [2,4,1,3] wynik to 10 i 3 dodawania; Hanoi dla 3 krążków kończy po 7 ruchach. Te wartości potwierdzają osobne testy, lecz nie dodajemy ich do porównania potęgowania.

09 / Testowanie i wyniki

Podstawowa seria: **44/44 PASS**. Uruchomiono: 2026-10-10T13:23:04.005Z. Ta sama seria przeszła w przeglądarce. Testy interfejsu: **17/17 PASS**, Chromium 156.0.8078.4, 2026-10-10T13:25:14.777Z. Wykonano je z zablokowanymi żądaniami HTTP/HTTPS.

Wejście / sprawdzenie	Oczekiwane i uzyskane	Status
Suma []	0	PASS
Suma [-7]	−7	PASS
Suma [2,4,1,3]	10	PASS
Obie potęgi: 0^0 i 5^0	1	PASS
Obie potęgi: $(-7)^1$	−7	PASS
Obie potęgi: 2^5 i $(-2)^5$	32 i −32	PASS
Hanoi n = 0, 1, 3, 8	0, 1, 7, 255 ruchów; całość na C	PASS
Suma po pierwszym powrocie	Stos [#1,#2]; #3 = 2, finished	PASS
Hanoi po pierwszym ruchu	A=[3,2], B=[], C=[1]; stos [1,2,3]	PASS
Niepoprawny zakres i ułamki	Wejście odrzucone	PASS
Większy na mniejszym / duplikat / brak	Stan odrzucony	PASS
Wszystkie metody po zakończeniu	Pusty stos, finished, metryki zgodne	PASS
Cała dziedzina potęgowania	462 zgodne przebiegi, ** jako oracle	PASS

Tabela 3. Skrót pokrycia. Każdy z 44 testów ma osobno nazwę, wejście, oczekiwane, uzyskane i status w wyniki/testy.json oraz testy.txt. Taką tabelę wyświetla też aplikacja.

Dwa stany pośrednie i niezależna kontrola

Nie wystarcza sam wynik końcowy. Pierwszy test pośredni potwierdza usunięcie zakończonej ramki sumy i zachowanie wyniku w drzewie. Drugi sprawdza pierwszy ruch Hanoi, układ słupków oraz id aktywnego wywołania. Oczekiwania są zapisane jawnie, niezależnie od rendererów. Dla potęg oracle jest operator **; dla Hanoi sprawdzane są $2^n - 1$ ruchów, docelowa wieża i legalność po każdym ruchu.

Kontrole interfejsu i naprawione problemy środowiska

17 kontroli obejmuje otwarcie file://, Krok, wyróżnienie kodu, Reset, własne i błędne dane, scenariusze, Start/Pauza/tempo, dobre i złe odpowiedzi, oba warianty potęgi, Hanoi, tabelę, testy przeglądarkowe, widok mobilny i brak błędów JavaScript. Pierwsze uruchomienie narzędzia Chromium ujawniło brak bibliotek systemowych; rozpakowano wymagane biblioteki w tymczasowym katalogu bez zmiany konfiguracji serwera. Nie był to błąd algorytmu. W wykonanych seriach nie wykryto niezgodnych wyników domenowych.

10 / Podsumowanie, źródła i AI

Wnioski

Przypadek bazowy zatrzymuje zejście, ale nie kończy całej pracy: wyniki są składane podczas powrotów. Stos pokazuje aktywną ścieżkę, a drzewo pełną historię wywołań. Szybkie potęgowanie redukuje wzrost czasu i stosu z liniowego do logarytmicznego. Hanoi pokazuje, że niewielki stos nie oznacza małej liczby operacji: głębokość jest liniowa, lecz liczba ruchów wykładnicza.

Ograniczenia i rozwój

Zakresy wejścia są celowo ograniczone kartą. Rejestr zdarzeń powstaje przed odtwarzaniem, więc aplikacja nie jest debuggerem wstrzymującym rzeczywisty stos silnika JavaScript. Rekonstrukcja stanu jest jednak zgodna z prawdziwymi zdarzeniami wykonania i sprawdzana testami. Przebudowa drzewa DOM jest czytelna i prosta, lecz dla dużych śladów wymagałaby aktualizacji przyrostowej lub wirtualizacji. Możliwy rozwój to generatory zdarzeń na żądanie i potęgowanie modularne z BigInt; nie są one częścią tej pracy.

Wykorzystanie narzędzi AI i wkład

Użyto OpenAI Codex przez OpenClaw do analizy karty, przygotowania kodu, testów, źródeł dokumentów, scenariusza obrony i weryfikacji. Kod oraz materiały tego pakietu powstały przy istotnym udziale AI. Nie deklarujemy, że student samodzielnie napisał każdy plik lub przeprowadził opisane testy ręcznie. Rzeczywiste uruchomienia wykonał agent; ich wyniki zapisano w pakiecie.

Potwierdzony wkład Edvina: wybór i przekazanie zadania, miejsca zapisu i danych autora. Przed oddaniem należy przejrzeć kod, przećwiczyć obronę i dopisać tylko faktycznie wykonane własne zmiany. Gotowy produkt nie zastępuje samodzielnego wyjaśnienia implementacji.

Jak kontrolowano wynik AI

Sprawdzono kontrakty wejścia, jawne oczekiwania testów, wyczerpującą dziedzinę potęgowania, legalność Hanoi, dwa stany pośrednie i zgodność liczników. Interfejs uruchomiono bez sieci; z niego pochodzą podpisane zrzuty. Nie użyto filmu ani skopiowanej wizualizacji. Wyników nie zadeklarowano przed wykonaniem testów.

Materiały i odwołania

1. [1] Karta zadania: ASD1-M1-14.html, „Rekurencja krok po kroku”, Dziennik UwB, rok 2026/2027. Wymagania i zakres liczb.
2. [2] R. Sedgewick, K. Wayne, Princeton, „2.3 Recursion”, introcs.cs.princeton.edu/java/23recursion/, dostęp: 10.10.2026. Materiał pomocniczy dotyczący przypadków bazowych i Hanoi; nie wykorzystano gotowej wizualizacji.
3. [3] Własne artefakty projektu: app/algorithms.js, app/playback.js, tests/suite.js, tests/browser.cjs oraz wyniki/*.json. Źródło liczników, zrzutów i rzeczywistych wyników.

> Przy oddawaniu: aplikacja i kod + prezentacja + raport PDF. Osobno dodaj link repozytorium GitHub do pola „Linki” w Dzienniku. Repozytorium nie zastępuje plików. Termin: 26.10.2026, 23:59; obrona: 28.10.2026, 08:30–11:40 [1].